Obstacle-Aware Quadrotor Control using RRT* Planning and CLF-CBF-QP Control

Luisa Chavez
Department of Electrical and Computer Engineering
North Carolina State University
Raleigh, NC, USA
Email: lchavez@ncsu.edu

***Abstract*—This project moves the state of the art autonomous quadrotor navigation by integrating a 3-D RRT* global planner with a safety-critical controller based on Control Lyapunov Functions(CLF) and Higher-Order Control Barrier Funtionc (HOCBF). The planner yields a 3-D reference trajectory that is then realized through a quadratic program that imposes a CLF-based translation dynamics model that covers the entire 3D space, along with a second order CBF barrier constraint on the horizontal plane to ensure that the quadrotor avoids all obstacles. Despite, numerical convergence issues emerging for a full CBF barriers constraint encompassing the entire SE(3) dynamical system, this choice of CBF barrier constraint structure offers a good balance of safety and optimality. Simulation experiments conducted on a dense indoor setting illustrate that the CLF-CBF-QP-based controller satisfies safety, removes all instances of violations of clearance from obstacles, and tracks the reference trajectory exactly, but with negible PD baseline offset. Quantitative comparison made between the two methods illustrate that the CLF-CBF-QP based controller makes a significnatly better case for improving clearance from the obstacles, along with improved trajectory stability, as opposed to its PD cntroller baseline.**

I. Introduction

Quadrotor platform have benn develop as a prominent platform for reasearch and development activities, including applications such as inspection, package delivery, environmental surveillance, and search-and-rescue missions. The addes ability of hovering makes then apt for autonomous navigation. However, controlling such robots is a difficult task because of their underactuated, nonliar, and strongly soplued dynamics [1]. These issues get reinforced when operating in a chaotic environment that required tough constraints on avoiding obstacles and maintaining safety.

Traditional controllers such as Proportional-Derivative (PD) control [2], Proportional-Integral-Derivative (PID) control [3], and Linear Quadratic Regulators (LQR) [4] can achieve acceptable tracking performance in simplified or obstacle-free scenarios. Nevertheless, these approaches lack guarantees of stability and do not inherently enforce safety constraints. Therefore, classical feedback controllers may lead to unsafe behaviors, specifically when the UAV is required to navigate near obstacles or follow aggressive trajectories [5].

Control Lyapunov Functions (CLF) and Control Barrier Functions (CBF) have emerged as powerful tools for guaranteeing stability and safety within a unified optimization-based control framework [6], [7]. A CLF ensures convergence of the tracking error via Lyapunov stability [8], while CBF imposes forward-invariant safety conditions that prevent the system from entering to unsafe areas [5]. By formulating both conditions as constraints in a Quadratic Program (QP), one can synthesize control inputs that are stabilizing and safe at every time step [7].

It is necessary to implement the control level of safety, along with finding a possible path that can baypass the obstacles. For this purpose, the current study combines the samplig-based planning method of 'Rapidly-Exploring Random Trees' (RRT) [9], along with its optimal version 'RRT*' [10], to yield a collision-free 'global tajectory' for the quadrotor within its operational environment. The generated 'global trajectory' is finally smoothed through 'cubic splines' to be control by the CLF-CBF-QP controller.

For this purpose, a basci PD controller is compared to the CLF-CBF-QP method on a multi-obstacle trajectory provided by the RRT* system. The present system, developed using MATLAB and integrated with CoppeliaSim via the ZMQ API, shows better tracking smoothness and ability to avoid collisions, thus proving the feasibility of optimization-based techniques for aerial robots.

II. Methods

This section presents the simulation framework, quadrotor dynamical model, reference trajectory generation, the baseline PD controller, and the construction of the Control Lyapunov Function (CLF) and Control Barrier Function(CBF) constraints used in the CLF-CBF-QP controller.

A. Simulation Environment: CoppeliaSim

CoppeliaSim (formerly V-REP) is a widely used software for simulation in education and robotics research [11]. It follows a distributed architecture that allows robots to be controlled internally or externally through APIs in various languages [12]. In this project, CoppeliaSim simulates an indoor environment containing 14 small obstacles and 8 walls segments, each modeled as spheres with a predefined safety radius. The control algorithms run externally in MATLAB and communicate with the simulation in real time through the ZMQ Remote API, sending control inputs at 20Hz (50 ms).

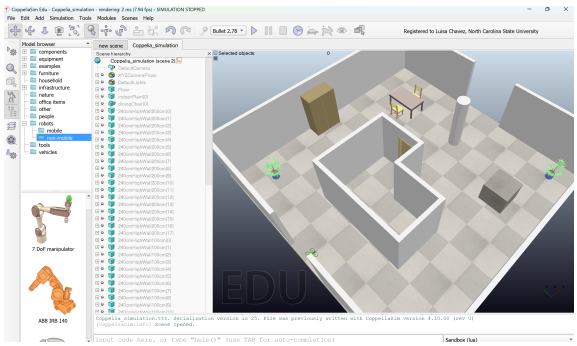

Figure 1. CoppeliaSim scenery

B. MATLAB Implementation

MATLAB is used to implement controllers and solve the embedded quadratic programs. As a high-level numerical computing environment, MATLAB supports matrix operations, spline interpolation, and real-time optimization, making it suitable for implementing model-based control laws [13]. Moreover, MATLAB offers a variety of toolboxes for control systems, dynamics systems, and robotic simulation, facilitating the integration of physical models [14].

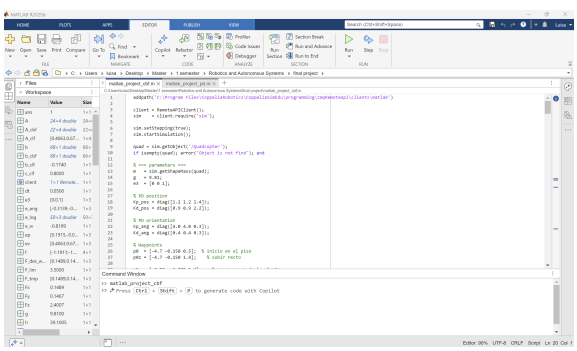

Figure 2. MATLAB software

To obtain continuous references trajectories, the discrete 3-D waypoints generated by the RRT* planner are interpolated using cubic splines, producing smooth reference position and velocity signals $p_{\text{ref}}(t)$ and $v_{\text{ref}}(t)$, which are used by both the PD controller and the CLF-CBF-QP controller.

C. Path Planning with RRT*

A sampling-based RRT* planner is used to generate a collision-free reference path in the 3D workspace. The state used by the planner is $p = [x\ y\ z]^\top$, constrained to the box

$$x \in [-5.5, 5.5]\ \text{m}, \quad y \in [-5.5, 5.5]\ \text{m}, \quad z \in [0.2, 1.8]\ \text{m}. \tag{1}$$

The start and goal configurations are

$$p_{\text{start}} = [-4.7,\ -0.15,\ 0.5]^\top, \qquad p_{\text{goal}} = [4.5,\ -0.125,\ 0.3]^\top. \tag{2}$$

Obstacles are modeled as spheres centered at the measured positions of the walls and furniture extracted from CoppeliaSim, with safety radius $r_{\text{safe}} = 0.8$ m. These spheres form a keep-out zone that is implemented in the collision checker of the RRT* algorithm to provide a buffer distance from each obstacle. RRT* starts by sampling a new candidate state from a uniform 3-D box specifies. It then connects this candidate to the nearest node if the line is free from collision. A goal bias of 0.1 is applied, such that a random number is generated, and if less than 0.1, the random candidate is replaced by the goal position.The most prominent parameters for RRT* are:

- Maximum number of iterations: $N_{\max} = 3000$,
- Step size: $\Delta s = 0.6$ m,
- Goal region radius: $r_{\text{goal}} = 0.6$ m,
- Rewiring neighborhood radius: $r_{\text{near}} = 1.0$ m.

The resulting sequence ofpoints $\{p_i\}_{i=1}^{N_w}$ is directly utilized as reference waypoints for generation of the 3D splines.

D. Quadrotor Dynamics

The quadrotor is modeled using a simplified translational model [2]:

$$\dot{\mathbf{p}} = \mathbf{v}, \tag{3}$$

$$\dot{\mathbf{v}} = \begin{bmatrix} u_x \\ u_y \\ -g + \frac{T}{m} \end{bmatrix}, \tag{4}$$

where u_x, u_y are lateral accelerations and T is the total thrust. This model follows the commonly used "flat-accelerations + thrust" approximation for controller prototyping.

Orientation evolves according to:

$$\dot{R} = R[\Omega]_\times, \tag{5}$$

$$J\dot{\Omega} = \tau - \Omega \times (J\Omega), \tag{6}$$

E. Baseline PD controller

The nominal tracking controller computes the commanded accelerations as:

$$u_{PD} = K_p\, e_p + K_d\, e_v, \tag{7}$$

with empirically tuned gains:

$$K_p = \text{diag}(1.2,\ 1.2,\ 1.4), \qquad K_d = \text{diag}(0.9,\ 0.9,\ 2.2). \tag{8}$$

Orientation is stabilized by a PD torque controller:

$$\tau_{PD} = K_{p,\text{ang}}\, e_{\text{ang}} + K_{d,\text{ang}}\,(-\Omega), \tag{9}$$

using:

$$K_{p,\text{ang}} = \text{diag}(4,\ 4,\ 0.3), \qquad K_{d,\text{ang}} = \text{diag}(0.4,\ 0.4,\ 0.3). \tag{10}$$

This PD controller is used as the nominal input for the CLF-CBF-QP controller.

F. Control Lyapunov Function (CLF)

The CLF is defined on the full 3D position and velocity errors:

$$e_p = p_{\text{ref}} - p, \qquad e_v = v_{\text{ref}} - v, \tag{11}$$

where $p_{\text{ref}}(t)$ and $v_{\text{ref}}(t)$ are obtained from the spline-based reference trajectory, and $a_{\text{ref}}(t)$ denotes its second derivative (acceleration). The translational dynamics can be written as

$$\dot{v} = f_v + G_v\, u, \tag{12}$$

with

$$f_v = \begin{bmatrix} 0 \\ 0 \\ -g \end{bmatrix}, \qquad G_v = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & \frac{1}{m} \end{bmatrix}, \qquad u = \begin{bmatrix} u_x \\ u_y \\ T \end{bmatrix}. \tag{13}$$

For this project the CLF used is:

$$V(e_p, e_v) = e_p^\top Q_p\, e_p + e_v^\top Q_v\, e_v, \tag{14}$$

with $Q_p = \text{diag}(1, 1, 1)$ and $Q_v = \text{diag}(0.5, 0.5, 0.5)$. Using the error dynamics

$$\dot{e}_p = e_v, \qquad \dot{e}_v = a_{\text{ref}} - f_v - G_v\, u, \tag{15}$$

the time derivative of V can be decomposed as

$$\begin{aligned} \dot{V} &= 2e_p^\top Q_p e_v + 2e_v^\top Q_v \big(a_{\text{ref}} - f_v - G_v u\big) \\ &= L_f V + L_g V\, u, \end{aligned} \tag{16}$$

where

$$L_f V = 2e_p^\top Q_p e_v + 2e_v^\top Q_v (a_{\text{ref}} - f_v), \tag{17}$$

$$L_g V = -\,2e_v^\top Q_v G_v. \tag{18}$$

The CLF constraint enforced in the QP is

$$L_f V + L_g V\, u \;\le\; -c_{\text{clf}} V + \delta, \tag{19}$$

with $c_{\text{clf}} = 0.8$ and relaxation variable $\delta \ge 0$.

G. Control Barrier Functions (CBF)

For each obstacle with center $\mathbf{p}_{\text{obs}}$ and safety radius r_{safe}, define:

$$h(\mathbf{p}) = \|\mathbf{p}_{xy} - \mathbf{p}_{\text{obs}}\|^2 - r_{\text{safe}}^2, \tag{20}$$

where $\mathbf{p}_{xy}$ denotes the horizontal position of the quadrotor.

Using a second-order HOCBF, the safety constraint is:

$$\ddot{h} + k_1\, \dot{h} + k_0\, h \ge 0, \tag{21}$$

with tuned gains:

$$k_0 = 6.0, \qquad k_1 = 4.5. \tag{22}$$

All obstacles use a safety radius of:

$$r_{\text{safe}} = 0.8 \text{ m}. \tag{23}$$

In practice, the barrier functions are defined on the horizontal position $p_{xy} = [x\ y]^\top$, using the projection of the obstacle centers onto the x–y plane. This effectively creates cylindrical keep-out regions around each obstacle in the horizontal plane.

H. CLF-CBF Quadratic Program (QP)

At each control step, the optimal input (u_x, u_y, T) is obtained by solving:

$$\min_{u,\,\delta} \quad \|u - u_{PD}\|^2 + \rho\, \delta^2 \tag{24}$$

$$\text{s.t.} \quad \text{CLF: } L_f V + L_g V\, T \;\le\; -c_{\text{clf}}\, V + \delta, \tag{25}$$

$$\text{CBF: } A_{\text{cbf}}\, u \;\le\; b_{\text{cbf}}, \tag{26}$$

$$\text{bounds: } |u_x|,\, |u_y| \le 2.5, \qquad 0 \le T \le 2mg. \tag{27}$$

The parameter $\rho = 10$ penalizes relaxation of the CLF constraint.

The quadratic program is solved using MATLAB's `quadprog`, and the resulting control input is sent to CoppeliaSim in real time.

III. Results

A. Reference Tracking Performance

Figure 3 showcases the tracking of the system along its x, y, and z axes. Both the PD and the CLF-CBF-QP controllers can follow the previously generated spline; however, QP possesses the following properties:

- In the x axis, the accuracy of both controllers is comparable for tracking.
- It can be observed that, for the y-axis, the QP controller is closer to the reference value as it moves among a densely clustered set of obstacles, compared to the PD controller.
- Along the z axis, the QP controller achieves smoother altitude profile with less oscillations, especially for drastic vertical changes.

These results provide evidence that the CLF term helps to stabilize the trajectory, and the constraints provides by CBF ensure that these deviations be made safely, if necessary..

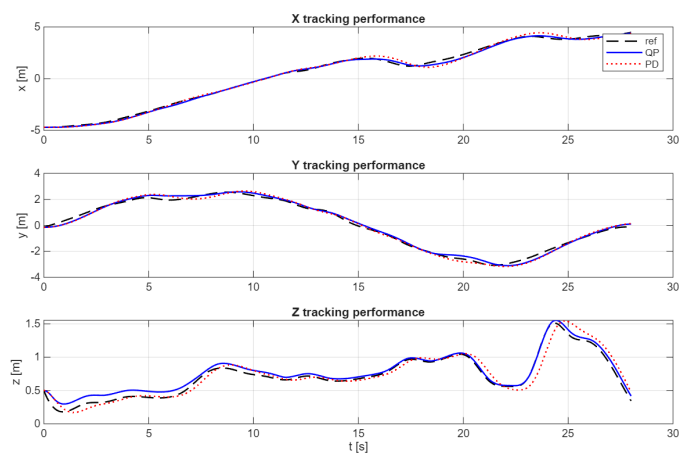

Figure 3. Tracking performance along the x, y, and z axes. The reference trajectory (black dashed) is compared with the CLF–CBF–QP (blue) and PD (red) responses.

B. Control Input Behavior

In Figure 4 shows the controls u_x, u_y, and u_z computed by the both controllers.

The PD and CLF–CBF–QP controllers provide approximately equivalent inputs for a given trajectory. However, the QP controller provides local corrective inputs:

- u_x and u_y closely follow the PD baseline, except for short periods of active CBF constraints.
- u_z remains smooth, although the QP controller exerts a mildly greater accuracy of vertical controls.

These results mean that a QP controller affects the PD controller's input in a minimal way, acting only if safety is a concern.

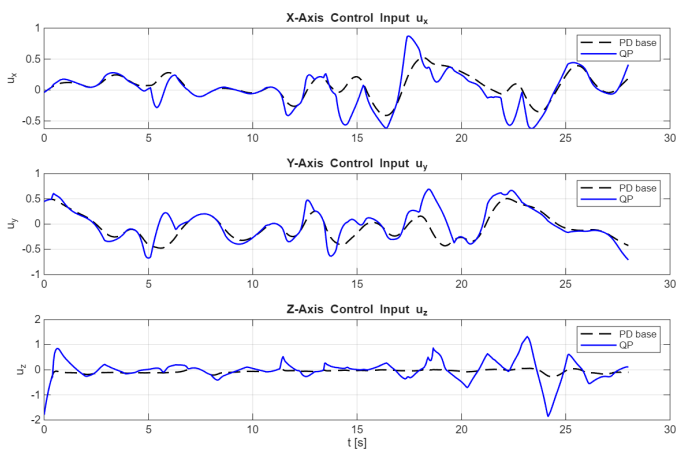

Figure 4. Control input components generated by the CLF–CBF–QP controller (solid blue) and the nominal PD controller (black dashed).

C. Deviation from PD Baseline

Figure 5 shows the norm difference $|u^*(t) - u_{PD}(t)|$. The deviation remains small during most of the trajectory and increases only when the quadrotor approaches obstacle regions. The peaks correspond to safety-enforcing actions from the CBF constraints.

- QP controller is comparable to a PD controller for most of the path.
- Corrections increase only if necessary to keep minimal clearance.
- It prevents any unnecessary or aggressive changes.

This evidence supports the interpretation of the CLF–CBF–QP controller as a safety filter.

D. Safety Analysis: Minimum Distance to Obstacles

Figure 6 shows the minimum distance between the quadrotor and the closest obstacle at each timestep. Key observations include:

- The CLF-CBF-QP controller always keeps a distance above the safety radius of $r_{\text{safe}} = 0.8\,\text{m}$,
- The PD controller often crosses this limit, entering into unsafe areas.

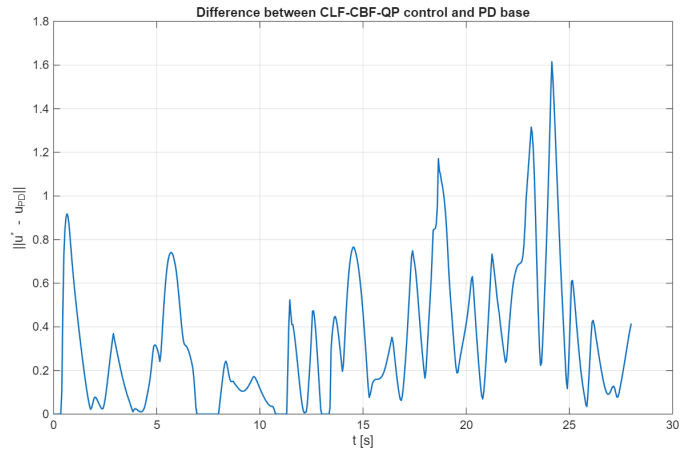

Figure 5. Norm of the control deviation between the optimal CLF–CBF–QP input u^* and the nominal PD control u_{PD}.

- The difference widens for cluttered scenes where the constraints of CBF lead to a significantly altered QP trajectory.

This is the strongest evidence of the advantage of integrating safety guarantees into the controller.

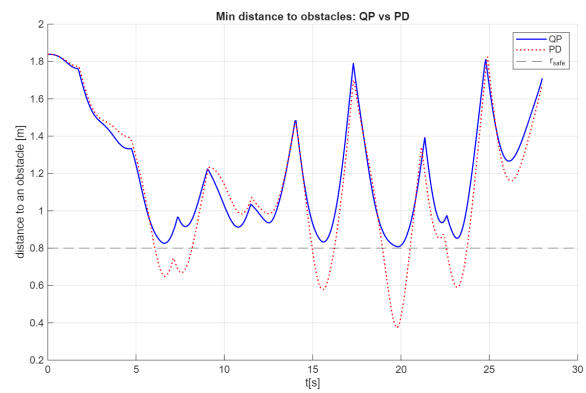

Figure 6. Minimum distance between the quadrotor and the closest obstacle under both controllers.

E. Position Error Norm

Figure 7 presents the evolution of the position error norm $\|e_p(t)\|$. The CLF–CBF–QP controller outperforms the PD controller throughout the entire trajectory:

- Lower peak error,
- Lower average deviation,
- Error variability is reduced in cluttered areas,
- Consistently smoother convergence.

The PD controller shows a high peak value because the controller lacks awareness of safety.

F. Global XY Trajectory Visualization

Figure 8 shows the full x–y projection of the trajectory, including inflated obstacle boundaries. The results reveal that:

- both controllers follow the general shape of the RRT* path,

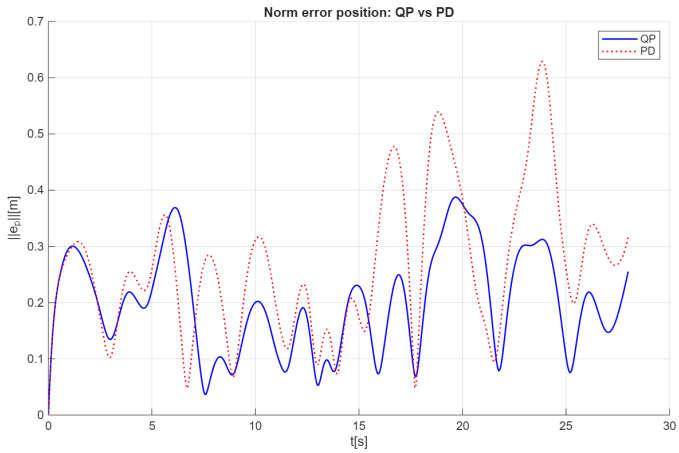

Figure 7. Time evolution of the position tracking error $\|e_p(t)\|$ for both controllers.

- the CLF–CBF–QP trajectory remains centered within safe corridors,
- the PD controller frequently intersects unsafe regions,
- the QP controller naturally takes wider curves near dense clusters, demonstrating intelligent avoidance behavior.

This visualization highlights the safety and robustness benefits of embedding CBF constraints into the controller.

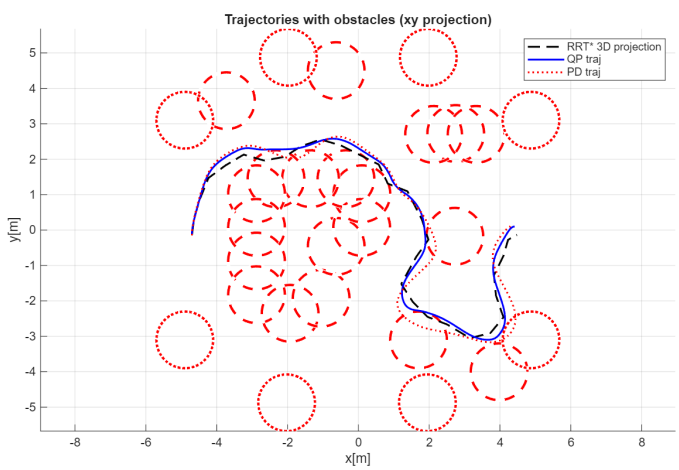

Figure 8. Planned RRT* path (black dashed), executed trajectory using the CLF–CBF–QP controller (blue), and trajectory using the PD controller (red). Obstacles are shown as inflated safety regions.

G. Quantitative comparison

To compare both controllers, Table I summarizes the root-mean-square errors (RMSE), total RMSE, minimum obstacle distance, and number of safety violations. While the RMSE values of both controllers are comparable, the QP-based controller achieves significantly better safety performance, maintaining a larger minimum clearance and avoiding all violations.

Table I
COMPARISON BETWEEN PD AND CLF–CBF–QP CONTROLLERS

Metric	CLF–CBF–QP	PD
RMSE total [m]	0.218	0.296
Minimum obstacle distance [m]	0.808	0.375
Safety violations	0	121
Behavior near clutter	Proactive avoid.	Reactive / unsafe

IV. DISCUSSION

Different design decisions had a great impact on the final performance of the navigation system architecture. Compared to previous attempts, the optimized system uses a full three-dimensional RRT* planner and a three-dimensional CLF for position control. Hence, a natural baseline trajectory is provided that contains admissible altitude changes, and the CLF can stabilize the system along the x, y, and z axes. This causes a smooth reference trajectory that can be flown without the optimization layer.

The most significant simplification was made for the safety term. Although a full 3-D control barrier function formulation was considered, finding a way to implement it was difficult. When working in a 3-D space, the task of avoiding objects requires calculation of both the gradient derivative and Lie derivative of a volumetric distance function, making such a problem's constraints much larger. Moreover, working in a environment with real objects and real shapes made it harder because in the code, my objects are seen as spheres and not an actual chair or wall. Experiments have also shown that a numerical issue or infeasibility was experienced when trying to implement a full set of 3-D CBF constraints to avoid objects. Restricting a 3-D CBF to a horizontal plane, defined by parameters (x, y), allowed for a simpler calculation of a volumetric distance from objects.

The resulting architecture provides a transparent explanation of its behavior, where the CLF affects nominal tracking behavior on all axes, and the CBF is active only if lateral distances to the obstacle decrease. These facts can be explained by the empirical observations made about the obtained results, where the controls u_x, u_y differ from the PD controller's behavior given basically only close to the obstacle, and u_z stays close to the nominal behavior given by the PD controller and CLF. The CBF works as a saelective safety filter, and its two-dimensional projection avoids overconstraint of the optimization problems.

On the whole, the choice of architectures, including full 3-D planning, 3-D CLF, and 2-D HOCBF, is a trade-off between model completeness and numerical robustness. The ultimate architecture is guaranteed to remain stable, interpretable, and able to provide obstacle-aware flight, even without a full 3-D CBF.

V. CONCLUSION

In this work, a complete quadrotor navigation system is offered, combining a 3-D RRT* global planner with a CLF–CBF–QP safety-critical controller. The planner provides

a smooth, fully 3-D reference trajectory, and the CLF is responsible for stabilizing the quadrotor along all its translation degrees of freedom. Real-time safety is provided through 2-D HOCBF constraints, expressed in the horizontal plane, that have been found sufficient for guaranteeing a safe passage of vehicle through a cluttered environment.

Experiments have confirmed that the QP controller makes a significant improvement to safety performance, without worsening the accuracy of tracking tasks compared to the nominal PD controller. The Quadratic Program controller acts much like a PD controller for unconstrained patters, making adjustments only when neccesary, thus resulting in fewer errors, smaller corrective movements, and a constant safety distance to obstacles.

One of the most important lessons learned from this investigation is that a balance between accuracy and numerical tractability for optimization-base control is a highly important aspect of its implementation. Although a full 3-D CBF was beyond the scope of this implementation, this design was able to achieve successful and safe passage. Future development works must include incorporating a 3-D CBF for the definition of the volume of obstacles, implementing a risk-aware planner to reduce interaction with high obstacles densities, or using sensors that provide more information about the surroundings, such as LIDAR.

References

[1] T. Emran, M. J. Nordin, and R. Ahmad, "A comprehensive review on quadrotor control and autonomous flight," *International Journal of Micro Air Vehicles*, vol. 10, no. 4, pp. 287–314, 2018.

[2] S. Bouabdallah, P. Murrieri, and R. Siegwart, "Design and control of a miniature quadrotor," in *Proceedings of the IEEE International Conference on Intelligent Robots and Systems (IROS)*, 2004, pp. 153–158.

[3] P. Castillo, R. Lozano, and A. Dzul, "Modeling and control of mini-flying machines," *Springer*, 2004.

[4] D. Gurdan, J. Stumpf, and M. e. a. Achtelik, "Energy-efficient autonomous four-rotor flying robot controlled at 1 khz," in *IEEE International Conference on Robotics and Automation (ICRA)*, 2007, pp. 361–366.

[5] A. D. Ames, S. Coogan, M. Egerstedt, G. Notomista, K. Sreenath, and P. Tabuada, "Control barrier functions: Theory and applications," *IEEE Control Systems Magazine*, vol. 39, no. 6, pp. 82–106, 2019.

[6] A. D. Ames, J. W. Grizzle, and P. Tabuada, "Control lyapunov functions and control barrier functions: A unified framework for safety critical control," in *Proc. 53rd IEEE Conference on Decision and Control (CDC)*, 2014, pp. 4208–4215.

[7] A. D. Ames, X. Xu, J. W. Grizzle, and P. Tabuada, "Control barrier function based quadratic programs for safety critical systems," *IEEE Transactions on Automatic Control*, vol. 62, no. 8, pp. 3861–3876, 2017.

[8] H. K. Khalil, *Nonlinear Systems*, 3rd ed. Prentice Hall, 2002.

[9] S. M. LaValle, "Rapidly-exploring random trees: A new tool for path planning," Iowa State University, Tech. Rep. TR 98-11, 1998.

[10] S. Karaman and E. Frazzoli, "Sampling-based algorithms for optimal motion planning," *The International Journal of Robotics Research*, vol. 30, no. 7, pp. 846–894, 2011.

[11] E. Rohmer, S. P. N. Singh, and M. Freese, "Coppeliasim (formerly v-rep): a versatile and scalable robot simulation framework," in *Proceedings of the 2013 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2013, pp. 1321–1326, www.coppeliarobotics.com.

[12] "Coppeliasim," https://en.wikipedia.org/wiki/CoppeliaSim, accessed: 2025-11-27.

[13] Wikipedia contributors, "MATLAB — Wikipedia, The Free Encyclopedia," https://en.wikipedia.org/wiki/MATLAB, [Online; accessed 27-Nov-2025].

[14] The MathWorks, Inc., "Robotics - mathworks," https://www.mathworks.com/solutions/robotics.html, 2025, [Online; accessed 27-Nov-2025].